Smart To-Do Manager

A Complete Cloud Computing & DevOps Project

Python · Flask · SQLite · Docker · Kubernetes · Jenkins · Terraform · ngrok · GitHub Webhooks

1. Project Overview

Smart To-Do Manager is a full-stack web application built with **Python/Flask** backed by an **SQLite** database. Its primary goal is not just to manage tasks — it is a live, working demonstration of a modern DevOps pipeline. Every single time a developer pushes code to GitHub, the application is automatically rebuilt as a Docker image, the old container is replaced, and the updated app is redeployed on a local Kubernetes cluster — all without touching the server manually.

The project covers the full DevOps lifecycle: source control → continuous integration → containerisation → orchestration → infrastructure as code. It is designed to be a portfolio-grade reference project for anyone learning Cloud & DevOps engineering.

End-to-End Automated Pipeline

```
git push → GitHub Repository → Webhook (HTTP POST) → ngrok public tunnel → Jenkins CI/CD →
Docker image build → Terraform (IaC) → Kubernetes deployment → Flask app running live
```

2. Technology Stack

Technology	Role in Project	Version / Notes
Python 3.10+	Application backend language	Core runtime
Flask	Lightweight WSGI web framework	Latest stable
SQLite	Relational database (file-based)	Built into Python stdlib
Jinja2	HTML templating engine (bundled w/ Flask)	Auto-escaping enabled
Git	Distributed version control	2.x
GitHub	Remote repository + webhook host	Cloud / Free tier
Jenkins	CI/CD automation server	LTS (Long-Term Support)
Docker	Containerise the Flask app	Engine 24.x+
Kubernetes	Orchestrate containers at scale	v1.28+
Minikube	Single-node local K8s cluster	v1.32+
kubectl	CLI to manage Kubernetes	Matches cluster version
Terraform	Infrastructure as Code (IaC)	v1.6+
ngrok	Expose localhost Jenkins to public internet	v3 Agent
Bash / Shell	Glue scripts in Jenkinsfile pipeline	POSIX sh

3. Features

- ✓ **Add Tasks** – type a task in the input box and hit Add; stored immediately in SQLite.
- ✓ **Delete Tasks** – remove any task with a single click; row deleted from DB.
- ✓ **Mark Complete** – toggle a task done/undone; status persisted in DB.
- ✓ **Responsive UI** – CSS Grid/Flexbox layout; works on mobile, tablet, and desktop.
- ✓ **SQLite Storage** – persistent, file-based DB; zero external DB server required.
- ✓ **Docker Container** – entire app packaged in a minimal Alpine-based image.
- ✓ **Kubernetes Deployment** – runs as K8s pods managed by a Deployment resource.
- ✓ **CI/CD Pipeline** – Jenkins automates build, test, and deploy on every push.
- ✓ **GitHub Webhooks** – push event fires the pipeline automatically.
- ✓ **Infrastructure as Code** – Terraform provisions Kubernetes resources declaratively.

4. Project Structure & File Explanations

Below is every file and directory in the repository with a plain-English explanation of what it does and why it exists.

Directory Tree

```
To-Do/ | +-- app/ # Flask application package | +-- app.py # Entry point – routes, DB logic | +-- requirements.txt # Python package dependencies | +-- templates/ | | +-- index.html # Jinja2 HTML template (UI) | +-- static/ | +-- style.css # Custom CSS stylesheet | +-- kubernetes/ # Kubernetes manifest files | +-- deployment.yaml # Defines Pods + ReplicaSet | +-- service.yaml # Exposes app via NodePort | +-- terraform/ # Infrastructure as Code | +-- main.tf # Terraform config (provider + resources) | +-- Dockerfile # Instructions to build Docker image +-- Jenkinsfile # Declarative CI/CD pipeline definition +-- .gitignore # Files Git should not track +-- README.md # Project documentation
```

app/app.py — Flask Application Entry Point

- Imports Flask, SQLite3, and initialises the app object.
- Defines the SQLite database path (e.g. `todo.db`) and creates the **tasks** table on first run.
- Route `GET /` — fetches all tasks from DB, renders **index.html**.
- Route `POST /add` — reads form data, inserts new task row, redirects to `/`.
- Route `GET /delete/<id>` — deletes task by primary key, redirects.
- Route `GET /complete/<id>` — toggles **done** boolean, redirects.
- Runs with `app.run(host='0.0.0.0', port=5000)` so it binds all interfaces (needed inside Docker).

app/requirements.txt — Python Dependencies

- Declares every Python package the app needs: `Flask`, `gunicorn` (production WSGI), etc.
- Used by `pip install -r requirements.txt` in both local setup and the Docker build step.
- Pinning versions (e.g. `Flask==3.0.3`) ensures reproducible builds across environments.

app/templates/index.html — Jinja2 HTML Template

- Extends Flask's Jinja2 templating — uses `{{ variable }}` and `{% for %}` blocks.
- Loops over the list of tasks returned by the route and renders each as a list item.
- Contains an HTML form (POST to `/add`) with a text input for new tasks.
- Each task has Delete and Complete anchor links pointing to the respective routes.
- Links to `static/style.css` via Flask's `url_for('static', filename='style.css')`.

app/static/style.css — Stylesheet

- Provides the visual design: colour scheme, fonts, spacing, and button styles.
- Uses CSS Flexbox/Grid to make the layout responsive on all screen sizes.
- Completely separate from backend logic — can be updated without touching Python.

Dockerfile — Container Build Instructions

- `FROM python:3.10-slim` — starts from a minimal Debian-based Python image (~50 MB).
- `WORKDIR /app` — sets the working directory inside the container.
- `COPY app/requirements.txt .` then `RUN pip install -r requirements.txt` — installs deps as a separate layer (Docker caches this).
- `COPY app/ .` — copies all application source files into the container.
- `EXPOSE 5000` — documents that the container listens on port 5000.
- `CMD ["python", "app.py"]` — default command that runs when the container starts.

kubernetes/deployment.yaml — Kubernetes Deployment

- `apiVersion: apps/v1 / kind: Deployment` — declares a Deployment resource.
- `spec.replicas: 2` — runs 2 identical Pod replicas for basic high-availability.
- `spec.selector.matchLabels` — ties the Deployment to Pods with matching labels.
- `spec.template.spec.containers[].image: cloud-devops-app:latest` — the Docker image to pull.
- `containerPort: 5000` — the port Flask listens on inside the Pod.
- Kubernetes will automatically restart any Pod that crashes (self-healing).

kubernetes/service.yaml — Kubernetes Service

- `kind: Service / spec.type: NodePort` — exposes the Pods on a static port of every cluster node.
- `spec.selector` — must match the labels in deployment.yaml so traffic routes correctly.
- `spec.ports[].port: 80 / targetPort: 5000 / nodePort: 30001` — maps external port 30001 → container port 5000.
- Access the app via `http://<minikube-ip>:30001` after applying this manifest.

terraform/main.tf — Infrastructure as Code

- Declares the **kubernetes** Terraform provider so Terraform can talk to the cluster.
- `resource "kubernetes_deployment"` — mirrors the deployment.yaml but managed by Terraform state.
- `resource "kubernetes_service"` — mirrors service.yaml; Terraform tracks drift and can re-apply.
- Allows the entire infrastructure to be version-controlled, reviewed, and reproduced with three commands: `init → plan → apply`.

Jenkinsfile — CI/CD Pipeline Definition

- Uses Jenkins **Declarative Pipeline** syntax (recommended over Scripted).
- Stage **Clone**: `git url: '...'` — checks out latest code from GitHub.
- Stage **Build**: `docker build -t cloud-devops-app .` — builds fresh Docker image.
- Stage **Clean**: `docker stop todo || true` and `docker rm todo || true` — removes old container safely.
- Stage **Deploy**: `docker run -d -p 5000:5000 --name todo cloud-devops-app` — starts new container.
- Stage **Infra** (optional): `terraform apply -auto-approve` — updates K8s resources via Terraform.
- `post { failure { ... } }` — sends alert or cleans up on pipeline failure.

.gitignore — Git Exclusion List

- Prevents committing `__pycache__/`, `*.pyc`, `venv/`, `todo.db`, `.terraform/` and `*.tfstate`.
- Keeps the repo clean and avoids accidentally pushing secrets or binary build artefacts.

5. Installation & Local Setup

Prerequisites

- Python 3.10 or higher installed (`python3 --version`)
- Git installed (`git --version`)
- pip package manager (`pip3 --version`)
- Docker Engine installed and running (`docker --version`)
- kubectl CLI (`kubectl version --client`)
- Minikube (`minikube version`)
- Terraform (`terraform -version`)
- Jenkins (installed, service running on port 8080)
- ngrok account + agent installed

Step 1 – Clone the Repository

Downloads the full project from GitHub into a local folder named To-Do.

```
git clone https://github.com/Vishwajeetsinghh01/To-Do.git cd To-Do
```

Step 2 – Create a Python Virtual Environment

Creates an isolated Python environment so project dependencies do not conflict with your system Python.

```
python3 -m venv venv
```

Step 3 – Activate the Virtual Environment

Your shell prompt will change to show (venv) when activated. All pip installs now go into this environment only.

```
# Linux / macOS source venv/bin/activate # Windows (Command Prompt) venv\Scripts\activate.bat # Windows (PowerShell) venv\Scripts\Activate.ps1
```

Step 4 – Install Python Dependencies

Reads requirements.txt and installs Flask and all other listed packages into the virtual environment.

```
pip install -r app/requirements.txt
```

Step 5 – Run the Flask Application Locally

Flask starts a development server. Open your browser and visit http://localhost:5000 to use the app.

```
cd app python3 app.py # Expected output: # * Running on http://0.0.0.0:5000 # * Debug mode: on
```

Step 6 – Deactivate Virtual Environment (when done)

Returns your shell to the system Python environment.

```
deactivate
```

6. Docker – Containerisation

Docker packages the Flask app and all its dependencies into a portable, self-contained image. The same image runs identically on any machine that has Docker — no more "works on my machine" problems.

6.1 Dockerfile Explained

```
FROM python:3.10-slim # Base image: minimal Python on Debian WORKDIR /app # All subsequent commands run here COPY app/requirements.txt . # Copy dep file first (layer cache optimisation) RUN pip install -r requirements.txt # Install deps (cached if file unchanged) COPY app/ . # Copy application source EXPOSE 5000 # Document that the app listens on 5000 CMD ["python", "app.py"] # Default start command
```

6.2 Build the Docker Image

Docker reads the Dockerfile line by line and creates a layered image. Each RUN/COPY creates a new layer.

```
# Run from the project root (where Dockerfile lives) docker build -t cloud-devops-app . # -t cloud-devops-app = tag/name for the image # . = build context is current directory
```

6.3 Run the Container

```
docker run -d -p 5000:5000 --name todo cloud-devops-app # -d = detached (background) # -p 5000:5000 = map host port 5000 → container port 5000 # --name todo = give container a human-readable name # cloud-devops-app = image to use
```

6.4 Verify and Inspect

```
docker ps # List running containers docker logs todo # View application stdout/stderr docker exec -it todo /bin/sh # Open shell inside container docker stop todo # Stop the container docker rm todo # Remove the container docker images # List all local images docker rmi cloud-devops-app # Remove the image
```

6.5 Rebuild After Code Change

Jenkins automates exactly these three commands in the CI/CD pipeline.

```
docker stop todo && docker rm todo docker build -t cloud-devops-app . docker run -d -p 5000:5000 --name todo cloud-devops-app
```

7. Kubernetes Deployment

Kubernetes (K8s) is a container orchestration platform. It manages where and how many container instances run, automatically restarts crashed pods, and provides networking between services. **Minikube** gives us a single-node local K8s cluster for development.

7.1 Start Minikube

Minikube spins up a VM or Docker container that acts as a single Kubernetes node.

```
minikube start # With specific driver (recommended): minikube start --driver=docker # Verify cluster is running: kubectl cluster-info minikube status
```

7.2 Load Local Docker Image into Minikube

This step is critical — without it, K8s cannot find the image when creating Pods.

```
# Minikube has its own Docker daemon — load the image into it minikube image load cloud-devops-app:latest # OR build directly inside Minikube's Docker: eval $(minikube docker-env) docker build -t cloud-devops-app .
```

7.3 Apply the Deployment Manifest

Kubernetes reads deployment.yaml and creates a Deployment object that manages 2 Pod replicas.

```
cd kubernetes kubectl apply -f deployment.yaml # Expected output: # deployment.apps/todo-deployment created
```

7.4 Apply the Service Manifest

The Service creates a stable network endpoint and NodePort so you can reach the app from outside the cluster.

```
kubectl apply -f service.yaml # Expected output: # service/todo-service created
```

7.5 Verify Deployment

```
kubectl get deployments # Deployment status kubectl get pods # Pod list + status kubectl get svc # Services kubectl describe pod # Full pod details kubectl logs # Pod logs (Flask output)
```

7.6 Access the Application

```
# Get Minikube IP minikube ip # Open app directly (opens browser automatically) minikube service todo-service # OR manually: http://:30001
```

7.7 Scale the Deployment

Kubernetes redistributes traffic across all replicas automatically via the Service.

```
# Scale up to 4 replicas kubectl scale deployment todo-deployment --replicas=4 # Scale back down kubectl scale deployment todo-deployment --replicas=2 # Verify kubectl get pods
```

7.8 Roll Out an Update

```
# After rebuilding image kubectl rollout restart deployment/todo-deployment kubectl rollout status deployment/todo-deployment # Rollback if needed kubectl rollout undo deployment/todo-deployment
```

7.9 Clean Up

```
kubectl delete -f deployment.yaml kubectl delete -f service.yaml minikube stop minikube delete
```

8. Terraform – Infrastructure as Code

Terraform lets you define your infrastructure (Kubernetes Deployments, Services, cloud resources) in declarative **.tf** config files and track them in version control. The key advantage: *your infrastructure is reproducible, reviewable, and auditable.*

8.1 main.tf Structure Overview

```
# Provider block — tells Terraform to use the Kubernetes provider terraform { required_providers { kubernetes = { source = "hashicorp/kubernetes" version = "~> 2.0" } } } provider "kubernetes" { config_path = "~/.kube/config" # Uses local kubeconfig } # Deployment resource resource "kubernetes_deployment" "todo" { metadata { name = "todo-deployment" } spec { replicas = 2 selector { match_labels = { app = "todo" } } template { metadata { labels = { app = "todo" } } spec { container { name = "todo" image = "cloud-devops-app:latest" port { container_port = 5000 } } } } } } # Service resource resource "kubernetes_service" "todo" { metadata { name = "todo-service" } spec { type = "NodePort" selector = { app = "todo" } port { port = 80 target_port = 5000 node_port = 30001 } } }
```

8.2 Initialise Terraform

Run this once when you first clone the repo or add a new provider.

```
cd terraform terraform init # Downloads the kubernetes provider plugin # Creates .terraform/ directory (do NOT commit this)
```

8.3 Preview Changes

Always run plan before apply. It is your diff/dry-run for infrastructure.

```
terraform plan # Shows exactly what Terraform will create/update/delete # No changes are made at this step — safe to run anytime
```

8.4 Apply Infrastructure

Terraform creates or updates resources to match the declared state in main.tf.

```
terraform apply # Type 'yes' when prompted # OR skip confirmation prompt: terraform apply -auto-approve
```

8.5 Inspect State

Terraform stores state in terraform.tfstate — never delete this file.

```
terraform show # Full state terraform state list # List all managed resources terraform state show kubernetes_deployment.todo
```

8.6 Destroy Infrastructure

```
terraform destroy # OR terraform destroy -auto-approve # Removes ALL resources declared in main.tf
```

9. Jenkins CI/CD Pipeline

Jenkins is an open-source automation server. It watches for GitHub webhook events and runs the pipeline defined in **Jenkinsfile** automatically. No manual steps are needed once Jenkins is configured.

9.1 Install Jenkins (Ubuntu / Debian)

```
# Add Jenkins repository and key curl -fsSL https://pkg.jenkins.io/debian/jenkins.io-2023.key \ | sudo tee /usr/share/keyrings/jenkins-keyring.asc > /dev/null echo 'deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \ https://pkg.jenkins.io/debian binary/' \ | sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null sudo apt-get update sudo apt-get install -y jenkins # Jenkins requires Java sudo apt-get install -y openjdk-17-jdk
```

9.2 Start & Enable Jenkins

```
sudo systemctl start jenkins sudo systemctl enable jenkins # Auto-start on boot sudo systemctl status jenkins # Verify it's running # Jenkins is now available at http://localhost:8080
```

9.3 First-Time Jenkins Setup

Install the Git, Docker Pipeline, and GitHub plugins during setup.

```
# Get the initial admin password sudo cat /var/lib/jenkins/secrets/initialAdminPassword # Paste it at http://localhost:8080 # Install suggested plugins when prompted # Create an admin user
```

9.4 Give Jenkins Docker Permission

Without this step, docker build commands in the pipeline will fail with a permissions error.

```
# Add jenkins user to docker group so it can run docker commands sudo usermod -aG docker jenkins # Restart Jenkins for the change to take effect sudo systemctl restart jenkins
```

9.5 Create a New Pipeline Job

```
# In Jenkins UI: # Dashboard → New Item → Enter name: todo-pipeline # Select: Pipeline → OK # Under Pipeline section: # Definition: Pipeline script from SCM # SCM: Git # Repository URL: https://github.com/Vishwajeetsinghh01/To-Do.git # Script Path: Jenkinsfile # Save
```

9.6 Complete Jenkinsfile

```
pipeline { agent any environment { IMAGE_NAME = 'cloud-devops-app' CONTAINER_NAME = 'todo' } stages { stage('Clone Repository') { steps { git branch: 'main', url: 'https://github.com/Vishwajeetsinghh01/To-Do.git' } } stage('Build Docker Image') { steps { sh 'docker build -t $IMAGE_NAME .' } } stage('Stop & Remove Old Container') { steps { sh ''' docker stop $CONTAINER_NAME || true docker rm $CONTAINER_NAME || true ''' } } stage('Deploy New Container') { steps { sh 'docker run -d -p 5000:5000 --name $CONTAINER_NAME $IMAGE_NAME' } } stage('Apply Terraform Infrastructure') { steps { dir('terraform') { sh 'terraform init' sh 'terraform apply -auto-approve' } } } } post { success { echo 'Deployment successful! App running at http://localhost:5000' } failure { echo 'Pipeline failed. Check logs above.' sh 'docker stop $CONTAINER_NAME || true' } } }
```

9.7 Trigger a Manual Build

```
# In Jenkins UI: # Open todo-pipeline → Build Now # Click the build number → Console Output to watch live logs
```

9.8 Useful Jenkins Commands

```
sudo systemctl restart jenkins # Restart Jenkins service sudo systemctl stop jenkins # Stop Jenkins
journalctl -u jenkins -f # Stream Jenkins service logs sudo cat /var/log/jenkins/jenkins.log #
Jenkins log file
```

10. GitHub Webhook Automation via ngrok

Jenkins runs on **localhost:8080** — GitHub cannot reach it directly because your machine doesn't have a public IP. **ngrok** creates a secure public HTTPS tunnel to your localhost, allowing GitHub's webhook POST requests to reach Jenkins.

10.1 Install ngrok

```
# Download from https://ngrok.com/download # OR use snap: sudo snap install ngrok # Authenticate
with your account token ngrok config add-authtoken # Get your token from:
https://dashboard.ngrok.com/get-started/your-authtoken
```

10.2 Start the Tunnel

Keep this terminal open. If you restart ngrok, you get a new URL and must update the webhook.

```
# Open a new terminal window and run: ngrok http 8080 # ngrok will display something like: #
Forwarding https://a1b2c3d4.ngrok-free.app -> http://localhost:8080 # Copy the https://... URL —
this is your public Jenkins URL
```

10.3 Configure GitHub Webhook

GitHub will immediately send a test ping. You should see it appear in Jenkins or the ngrok dashboard.

```
# In your GitHub repository: # Settings → Webhooks → Add webhook # # Payload URL:
https://a1b2c3d4.ngrok-free.app/github-webhook/ # ^ trailing slash required! # Content type:
application/json # Secret: (optional but recommended) # Events: Just the push event # Active:
checked # → Click 'Add webhook'
```

10.4 Enable Webhook Trigger in Jenkins

```
# In Jenkins: # Open todo-pipeline → Configure # Under 'Build Triggers': # [x] GitHub hook trigger
for GITScm polling # Save
```

10.5 Monitor Webhook Traffic

```
# ngrok provides a local dashboard: open http://127.0.0.1:4040 # Shows all incoming requests,
headers, body, and response codes # Extremely useful for debugging webhook issues
```

10.6 Test the Full Pipeline

```
# Make any change to a file, then: git add . git commit -m 'test: trigger CI pipeline' git push
origin main # Watch Jenkins automatically start a new build within seconds
```

11. Full CI/CD Workflow – Step by Step

#	Actor	Action	Detail
1	Developer	git push origin main	Pushes new commit to GitHub
2	GitHub	Fires webhook POST request	Sends JSON payload to ngrok URL + /github-webhook/
3	ngrok	Forwards request to localhost	HTTP POST → http://localhost:8080/github-webhook/
4	Jenkins	Webhook received, build triggered	Pipeline starts automatically
5	Jenkins	Stage: Clone	Clones/pulls latest code from GitHub
6	Docker	Stage: Build	docker build -t cloud-devops-app .
7	Docker	Stage: Clean	docker stop todo && docker rm todo
8	Docker	Stage: Deploy	docker run -d -p 5000:5000 --name todo cloud-devops-app
9	Terraform	Stage: Infra (optional)	terraform apply -auto-approve (updates K8s state)
10	Kubernetes	Pods updated	K8s pulls image, replaces running pods gradually
11	Flask App	Live at localhost:5000	Updated application is serving traffic

12. Quick Reference – All Commands

Flask (Local Development)

```
python3 -m venv venv && source venv/bin/activate pip install -r app/requirements.txt cd app && python3 app.py # App runs at http://localhost:5000
```

Docker

```
docker build -t cloud-devops-app . docker run -d -p 5000:5000 --name todo cloud-devops-app docker ps # list running containers docker logs todo # view Flask output docker exec -it todo /bin/sh # shell into container docker stop todo && docker rm todo docker images docker rmi cloud-devops-app
```

Kubernetes (Minikube)

```
minikube start --driver=docker eval $(minikube docker-env) # point shell to minikube Docker docker build -t cloud-devops-app . # build inside minikube kubectl apply -f kubernetes/deployment.yaml kubectl apply -f kubernetes/service.yaml kubectl get pods kubectl get svc kubectl describe pod kubectl logs kubectl scale deployment todo-deployment --replicas=4 kubectl rollout restart deployment/todo-deployment kubectl rollout status deployment/todo-deployment kubectl rollout undo deployment/todo-deployment minikube service todo-service # open app in browser kubectl delete -f kubernetes/ minikube stop && minikube delete
```

Terraform

```
cd terraform terraform init terraform plan terraform apply terraform apply -auto-approve terraform show terraform state list terraform destroy -auto-approve
```

Jenkins

```
sudo systemctl start jenkins sudo systemctl stop jenkins sudo systemctl restart jenkins sudo systemctl status jenkins sudo cat /var/lib/jenkins/secrets/initialAdminPassword journalctl -u
```

```
jenkins -f # live logs
```

ngrok

```
ngrok config add-authtoken ngrok http 8080 # Dashboard: http://127.0.0.1:4040
```

Git

```
git clone https://github.com/Vishwajeetsinghh01/To-Do.git git add . git commit -m 'feat: describe your change' git push origin main git log --oneline git status
```

13. Learning Outcomes

- **Cloud Computing Fundamentals**
  - Containerisation, orchestration, infrastructure provisioning, and cloud-native design patterns.
- **DevOps Lifecycle**
  - End-to-end understanding from developer commit through automated deployment to running production app.
- **CI/CD Automation**
  - How to configure Jenkins declarative pipelines, trigger builds via webhooks, and handle failures.
- **Infrastructure as Code**
  - Using Terraform to declare, version-control, and reproducibly provision infrastructure.
- **Docker Containerisation**
  - Writing efficient Dockerfiles, managing images and containers, layer caching optimisation.
- **Kubernetes Orchestration**
  - Deployments, Services, pod scaling, rollouts, rollbacks, and self-healing behaviour.
- **GitHub Webhooks**
  - How webhooks work (HTTP callbacks), configuring them, and using ngrok for local development.
- **Flask Web Development**
  - Building a REST-like web app with routing, Jinja2 templates, and SQLite integration.

14. Troubleshooting

Problem: *Port 5000 already in use*

```
sudo lsof -i :5000 kill -9 # OR run Flask on a different port: export FLASK_RUN_PORT=5001
```

Problem: *Docker: permission denied*

```
sudo usermod -aG docker $USER newgrp docker # apply without logout # For Jenkins: sudo usermod -aG docker jenkins && sudo systemctl restart jenkins
```

Problem: *Minikube cannot pull image (ErrImagePull)*

```
# Must build inside minikube's Docker daemon eval $(minikube docker-env) docker build -t cloud-devops-app . # Also set imagePullPolicy: Never in deployment.yaml
```

Problem: *Jenkins webhook not triggering*

```
# 1. Verify ngrok is running and URL is correct # 2. Check GitHub webhook delivery logs (Settings→Webhooks→Recent Deliveries) # 3. Ensure Jenkins has 'GitHub hook trigger' enabled # 4. Check Jenkins is reachable: curl https:///github-webhook/
```

Problem: *Terraform: cannot connect to Kubernetes*

```
# Ensure kubeconfig is valid kubectl cluster-info # Set KUBECONFIG if needed export KUBECONFIG=~/.kube/config
```

Problem: *Flask app not accessible inside Docker*

```
# Ensure app.py binds to 0.0.0.0 not 127.0.0.1 app.run(host='0.0.0.0', port=5000) # 127.0.0.1
inside a container is unreachable from outside
```

15. Future Improvements

- → User Authentication — JWT tokens or OAuth2 (Google/GitHub login)
- → REST API layer — JSON endpoints for mobile apps or third-party integrations
- → Cloud deployment — AWS EKS, GCP GKE, or Azure AKS with a LoadBalancer Service
- → Monitoring — Prometheus metrics + Grafana dashboards for request rate, latency, errors
- → Logging — centralised log aggregation with ELK Stack (Elasticsearch + Logstash + Kibana)
- → React / Next.js frontend — replace Jinja2 with a modern SPA
- → Task categories, priorities, due dates, and reminder notifications
- → Email / push notification system via SendGrid or Firebase
- → Helm charts — package Kubernetes manifests for easier version-managed deployments
- → Multi-environment pipeline — dev → staging → production with manual approval gates
- → Automated testing stage in Jenkins (pytest unit + integration tests)
- → Docker Compose for local multi-service development

Author

Vishwajeet Singh

github.com/Vishwajeetsinghh01/To-Do

This project is developed for educational and learning purposes.